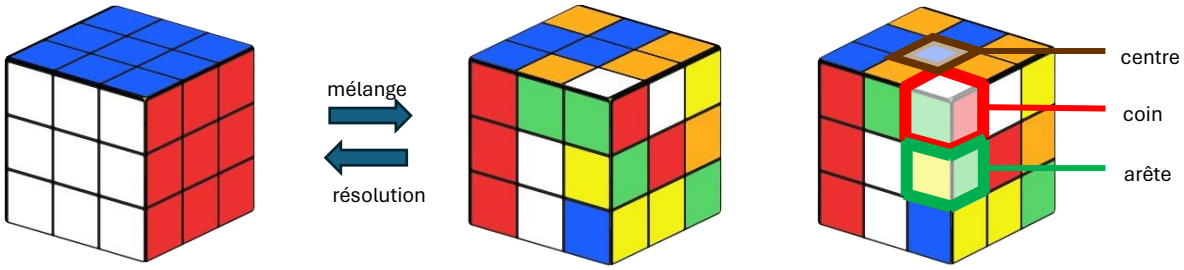
Recherche de solutions optimales au problème du Rubik's Cube

Le Rubik's Cube a été inventé en 1974 par Ernö Rubik, et est devenu depuis un casse-tête emblématique, notamment pour sa grande complexité. De nombreux algorithmes ont été élaborés pour permettre sa résolution, parmi lesquels certains cherchent à résoudre le Rubik's Cube de manière optimale, c'est-à-dire en un nombre minimal de mouvement. Cette approche rend le problème particulièrement ardu (il a été montré plus généralement que le problème de décision associé pour un Rubik's Cube n*n*n est NP-Complet [1]), et les algorithmes de résolution optimales sont limités dans leur capacité à résoudre certaines positions particulièrement éloignées de la position résolue. D'autres algorithmes se restreignent ainsi à la recherche de solution presque optimale, c'est-à-dire qui utilisent un nombre de mouvement proche du minimum requis.

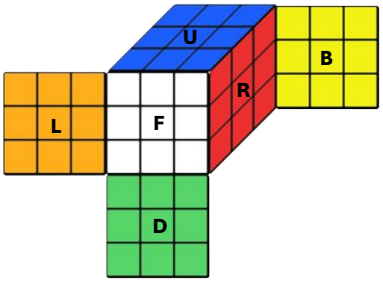
On cherche dans ce TIPE à comparer deux algorithmes de résolution, l'un optimal et l'autre presque optimal, selon la longueur des solutions qu'ils proposent, ainsi que leur complexité temporelle et spatiale.

I- Le Rubik's Cube et son groupe

Introduisons d'abord le casse-tête du Rubik's Cube plus en détails. Le Rubik's Cube est un cube dont les six faces sont coloriés de six couleurs distinctes dans l'état résolu. Il est composé de 27 sous-cubes secondaires (dont un au centre du cube), qui grâce au mouvement indépendant des faces, peuvent être mélangés. Résoudre le Rubik's Cube, c'est donc le ramener d'une position quelconque, où les cubes secondaires sont mélangés, à l'état résolu où les faces sont monocolores. On peut d'emblée distinguer trois types de cubes secondaires, selon leur nombre d'étiquettes : les « centres », au nombre de 6 (un pour chaque face) possèdent une étiquette, les arêtes, au nombre de 12, en possèdent deux, et enfin les 8 coins en possèdent trois. On peut considérer, et c'est ce que l'on fera par la suite que les centres sont immobiles et que seuls les coins et les arêtes se déplacent.



Pour faciliter la description du Rubik's Cube, on attribue une lettre à chaque face : F, U, L, R, D, B, comme Front, Up, Left, Right, Back et Down. On dénote alors les coins et arêtes par les lettres des faces qui les entourent. On définit alors 18 mouvements sur le Rubik's Cube, qui correspondent aux rotations d'un, deux, ou trois tours de chacune des six faces. On les note F, F2, F3, B, B2 etc. où la lettre désigne la face tournée et le numéro de le nombre de rotations, effectuées dans le sens horaire. Lorsque l'on parlera par la suite de distance en nombre de mouvements, ce seront ces mouvements, sauf mention contraire, que l'on s'autorisera.



Le Rubik's Cube peut également être vu comme un groupe, ce qui se révèle central dans l'élaboration d'algorithmes de résolution informatique. Posons d'abord

$$E \coloneqq \{\, ensemble\ des\ positions\ du\ Rubik's\ Cube \,\}$$

E est l'ensemble de *toutes* les positions du Rubik's Cube, c'est-à-dire non seulement celles accessibles par les mouvements légaux depuis la position résolue, mais aussi celles que l'on obtient en démontant puis remontant le Rubik's Cube. D'après notre description précédente du Rubik's Cube, seuls les coins et les arêtes se déplacent, et ces derniers ne peuvent être intervertis car ils ne possèdent pas le même nombre d'étiquettes. On peut donc voir chacune des positions du Rubik's Cube comme un couple d'une position des coins et d'une position des arêtes. Ainsi $E = E_C \times E_A$,

$$\text{où } \begin{cases} E_C \coloneqq \{ensemble\ des\ positions\ des\ coins\ du\ Rubik's\ Cube\} \\ E_A \coloneqq \{ensemble\ des\ positions\ des\ arêtes\ du\ Rubik's\ Cube\} \end{cases}.$$

On peut alors définir le groupe du Rubik's Cube $(G,\ \circ)$. Les éléments de G sont les mélanges, c'est-à-dire les applications de E dans E qui déplacent coins et arêtes, et $\circ$ la loi de composition. De même, on peut définir les groupes de mélanges de coins et d'arêtes G_C et G_A, de telle sorte que tout mélange de G se décompose en deux applications, la première agissant sur E_C et déplaçant les coins, la seconde sur E_A déplaçant les arêtes.

On peut encore préciser notre description de G. En effet, prenons par exemple les coins, on remarque que tout mélange de coins peut se décomposer en deux étapes : d'abord, on peut permuter les coins, puis ensuite les faire tourner chacun sur eux-mêmes pour leur donner une certaine orientation. On se convainc même aisément que cette décomposition est unique. C'est ce que traduit la phrase mathématique suivante :

$\forall g \in G,\ \exists!\ (\sigma,\ \rho) \in Perm_C \times Rot_C,\ \ g = \rho \circ \sigma,$

$$\text{où } \begin{cases} Perm_C \coloneqq \{g \in G \mid g\ ne\ modifie\ pas\ l'orientation\ des\ coins\} \\ Rot_C \coloneqq \{g \in G \mid g\ ne\ permute\ pas\ les\ coins\} \end{cases}.$$

Cela correspond à dire que $G_C = Perm_C \rtimes Rot_C$. Les éléments de $Perm_C$ correspondent aux permutations des 8 coins, donc $Perm_C \cong \mathfrak{S}_8$. Les éléments de Rot_C peuvent quant à eux être décrit par le nombre de rotations dont il faut faire tourner chacun des coins, qui possèdent trois orientations possibles, donc $Rot_C \cong \left(\frac{\mathbb{Z}}{3\mathbb{Z}}\right)^8$. On peut procéder de même pour que G_A, ce qui permet de prouver que $G_A = Perm_A \rtimes Rot_A$ avec $Perm_A \cong \mathfrak{S}_{12}$ et $Rot_A \cong \left(\frac{\mathbb{Z}}{2\mathbb{Z}}\right)^{12}$.

On peut a priori se demander l'intérêt de ce détour compliqué par les groupes. Tout d'abord, remarquons que l'on peut confondre les ensembles de mélanges et de positions *G* et *E*. Il suffit pour cela de considérer la bijection

$$\begin{aligned} \psi : G &\to E \\ g &\mapsto g(I) \end{aligned}$$

Grâce à notre décomposition de *G* en 4 sous-groupes et puisque l'on fait se correspondre mélanges et positions, cela signifie que l'on peut représenter toute position par 4 données : les orientations et permutations des coins, et orientations et permutations des arêtes. Cela sera central dans notre implémentation pratique du Rubik's Cube. Par ailleurs, l'approche du groupe du Rubik's Cube permet d'aboutir au théorème dit fondamental, qui donne une condition nécessaire et suffisante pour qu'une position soit légale, c'est-à-dire accessible depuis l'état résolu par les mouvements légaux :

$$\text{Une position est légale} \Leftrightarrow \begin{cases} \sum_{i=1}^{8} o_{c_i} = 0 \\ \sum_{i=1}^{12} o_{a_i} = 0 \\ \epsilon(\sigma_c) = \epsilon(\sigma_a) \end{cases}$$

où $(o_{c_i})_{1\leq i\leq 8}$, $(o_{a_i})_{1\leq i\leq 12}$ désignent les orientations des coins et arêtes par rapport la position résolue, et σ_c et σ_a les permutations des coins et des arêtes.

Cela permet d'évaluer la complexité de ce casse-tête qui possède donc de l'ordre de

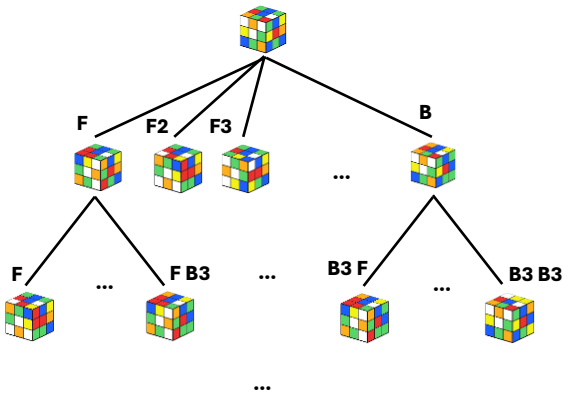
$$3^7 \times 2^{11} \times \frac{8!\ \times 12!}{2} \approx 4{,}3 \times 10^{19} \text{ positions.}$$

Enfin, c'est notamment en utilisant cette approche du groupe du Rubik's Cube, qu'une équipe de chercheurs est parvenue à démontrer en 2010 que toute position légale était résoluble en moins de 20 mouvements [2].

II- Les algorithmes de résolution optimale

Ramenons nous à présent notre problème initial : celui d'un algorithme capable de résoudre n'importe quelle position du Rubik's Cube en un nombre minimal de mouvements.

On peut d'abord être tenté d'emprunter une approche par force brute : pour avoir une solution optimale, il suffit en effet de parcourir toutes les séquences de mouvements au départ de la position à résoudre par longueur croissante, jusqu'à arriver à la position résolue. On peut visualiser cela dans un arbre dont la position à résoudre serait la racine, et dont tous les nœuds possèdent 18 fils, qui sont les 18 positions que l'on obtient depuis ce nœud par application des 18 mouvements. Bien entendu, plusieurs nœuds distincts peuvent correspondre à une même position dans un tel arbre.

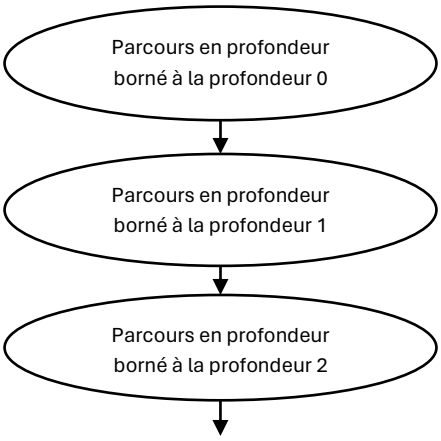


Trouver une solution optimale au Rubik's Cube, c'est alors simplement trouver un plus court chemin entre la racine et l'une des positions résolues qui apparaît dans sa descendance. Un parcours en largeur, qui explore les nœuds par profondeur croissante, suffit donc à trouver une solution optimale.

Cependant, l'arité importante des nœuds (qui est donc de 18) va nous limiter en pratique : d'une part, le nombre de nœuds et donc la complexité temporelle va croitre exponentiellement avec la profondeur de la position à résoudre, d'autre part, le nombre de nœud à stocker dans la queue nécessitée par le parcours en largeur et donc la mémoire utilisée par l'algorithme va elle aussi croître exponentiellement. Expérimentalement, on est limité à la profondeur 6 ou 7.

L'algorithme de Korf [3] va se baser sur ce même arbre, dont il faut trouver le plus court chemin entre la racine et l'une des positions résolues qui apparait dans sa descendance, mais en adoptant l'algorithme IDA* (Iterative Deepening A*) pour algorithme d'exploration, ce qui résout les problèmes de complexités spatiale et temporelle précédemment rencontrés.

Tout d'abord, l'aspect Iterative Deepening, ou approfondissement itératif, permet de résoudre le problème de complexité spatiale. Comme le nom le suggère, on va utiliser des parcours en profondeur, beaucoup moins coûteux spatialement que le parcours en largeur. On est cependant confrontés à un problème : puisque le parcours en profondeur ne parcourt pas les nœuds par profondeur croissante, comment assurer l'optimalité de la solution trouvée ? C'est là que l'aspect itératif rentre en jeu : on va effectuer une suite de parcours en profondeur borné à une certaine profondeur d'exploration, en augmentant progressivement cette borne jusqu'à trouver une solution.



On explore donc tous les nœuds à la profondeur 0, puis tous les nœuds à la profondeur 0 et 1, puis tous les nœuds à la profondeur 0, 1 et 2 etc. : de cette manière, l'ordre d'exploration se fait toujours par profondeur croissante. On assure ainsi l'optimalité de la solution trouvée, tout en limitant le nombre de nœuds à explorer dans la pile, qui sera dans le pire cas de 341.

Toutefois, on continue à explorer tous les nœuds d'un étage avant de passer à l'étage suivant, et la complexité temporelle reste pour l'instant exponentielle. C'est l'aspect A* qui va résoudre ce problème. On va construire des heuristiques admissibles, c'est-à-dire des fonctions qui prennent une position et donne une sous-estimation du nombre de mouvements nécessaires à sa résolution. Ainsi, soit h une telle heuristique, et imaginons que l'on explore un nœud p à la profondeur i lors du parcours en profondeur borné à la profondeur j : si $h(p) + i > j$, puisque l'heuristique h est admissible, cela signifie que l'on ne trouvera pas la position résolue dans la descendance de ce nœud lors de ce parcours en profondeur. Ainsi, on peut couper cette branche.

Pour construire de telles heuristiques, Korf propose de résoudre des « sous-problèmes » du Rubik's Cube. Par exemple, il propose l'heuristique suivante

$$h_2 \coloneqq \max(nb\ de\ mouvements\ pour\ résoudre\ les\ coins,$$
$$nb\ de\ mouvements\ pour\ résoudre\ 6\ des\ arêtes,\ nb\ de\ mouvements\ pour\ résoudre\ 6\ des\ autres\ arêtes)$$

Il est donc clair que cette heuristique est admissible. L'approche de Korf consiste alors à résoudre toutes les configurations de ces sous-problèmes préalablement, et à stocker les valeurs obtenues pour l'heuristique dans de grande bases de données [4]. Il apparaît ainsi que le choix de l'heuristique résulte d'un compromis entre complexité spatiale et temporelle : plus une heuristique permettra d'aller vite et donc de couper des branches, plus elle donnera d'informations et nécessitera de grande bases de données pour stocker ses valeurs. Korf conjecture ainsi en 1997 que le temps d'exécution de son algorithme est inversement proportionnel à la mémoire utilisée pour ses bases de données [3]. C'est ce que j'ai voulu vérifier dans ce TIPE, et j'ai pour cela construit 5 heuristiques qui possèdent un nombre croissant d'entrées dans leur bases de données.

Nom de l'heuristique	Nombre d'entrées dans les bases de données de l'heuristique
h_1	89 634 816
h_2	173 335 780
h_3	652 337 280
h_4	1 110 049 920
h_5	1 590 533 496

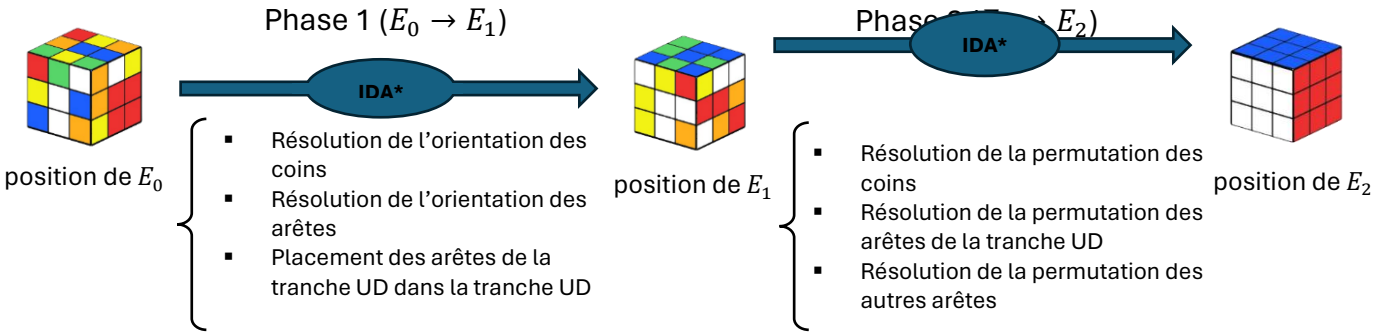
Enfin, l'algorithme « en deux-phases » de Kociemba [5] propose de relâcher la contrainte d'optimalité en cherchant des solutions presque optimales, c'est-à-dire qui utilisent un nombre de mouvements proche du minimum requis. On se ramène pour cela au groupe du Rubik's Cube en définissant une suite de sous-groupes décroissantes aux propriétés bien choisies. Posons

- $G_0 := < U, D, R, L, F, B >$ et $E_0 := \psi(G_0)$
- $G_1 := < U, D, R2, L2, F2, B2 >$ et $E_1 := \psi(G_1)$
- $G_2 := \{id\}$ et $E_2 := \psi(G_2)$

E_0 est l'ensemble des positions accessibles par les mouvements légaux depuis la position résolue, il s'agit donc de l'ensemble des positions légales. Il est également clair que l'ensemble E_2 est réduit à l'état résolu. Enfin, on peut prouver que les positions de E_1 vérifient les propriétés suivantes, qui sont centrales dans l'algorithme de Kociemba :

- Les coins et les arêtes des positions de E_1 sont bien orientés, c'est-à-dire orientés comme dans l'état résolu
- Les arêtes de la tranche UD (comprise entre les faces U et D) sont situées dans cette tranche

L'algorithme de Kociemba consiste alors à prendre la position à résoudre, une position quelconque de E légal, et à l'amener dans une première phase à une position de E_1, puis à amener cette nouvelle position à une position de E_2, c'est-à-dire à résoudre le Rubik's Cube.



Pour ne pas défaire le travail de résolution effectué dans la première phase, on s'assure dans la deuxième phase de toujours rester dans le sous-ensemble E_1 en utilisant que les mouvements de G_1. Les transitions de phase sont effectués avec l'algorithme IDA*, exactement comme pour l'algorithme de Korf, avec des heuristiques qui se basent sur la résolution de sous-problèmes, préalablement calculées et stockées dans des bases de données. Puisque l'on se force à passer par la position de E_1 la plus proche de la position à résoudre, on ne peut plus garantir l'optimalité de la solution trouvée ; cependant, il a été prouvé que le

nombre maximum de mouvements pour chacune des deux phases sont de 12 et 18, donc la solution obtenue par cette algorithme ne dépasse jamais 30 mouvements (et c'est en pratique très rare que ce maximum soit atteint), ce qui justifie l'appellation de résolution « presque optimale ».

III- Mise en pratique des algorithmes

Avant de pouvoir implémenter des algorithmes de résolution, on nécessite d'abord une représentation informatique du Rubik's Cube. Je me suis pour cela ramené à l'approche du groupe du Rubik's Cube en définissant la structure pos suivante pour représenter le Rubik's Cube en C, qui contient 4 tableaux pour les permutations et orientations des coins et arêtes.

```c
struct pos {
    int pc[8];
    int oc[8];
    int pa[12];
    int oa[12];
};
typedef struct pos pos;
```

Plus en détails, par exemple pour les coins, il a fallu d'abord attribuer à chacun des coins et numéro et une face de référence. Ensuite, pour un mélange quelconque des coins, voici à quoi peuvent ressembler les tableaux de permutations des coins pc (en haut) et d'orientations des coins oc (en bas).

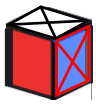
Indice	0	1	2	3	4	5	6	7
Valeur	5	6	7	4	0	1	3	2

Indice	0	1	2	3	4	5	6	7
Valeur	0	0	1	1	2	0	2	0

A l'indice i, pc donne le numéro du coin qui remplace le coin numéro i par ce mélange. De même, oc.(i) correspond à l'orientation du coin qui remplace le coin i par ce mélange, en définissant l'orientation selon les positions des faces marquées du coin remplacé et du coin qui le remplace, comme illustré ci-dessous.



Orientation : 0



Orientation :1



Orientation : 2

En noir : face marquée du coin remplacé

En rouge : face marquée du coin qui le remplace

On peut de la même manière décrire les 18 mouvements du Rubik's Cube par des tableaux de permutations et de réorientations. Puisque l'on confond mélanges et positions, cela nous donne alors des formules très simples pour l'application d'un mouvement à une position, qui correspond donc simplement à la composition de deux mélanges.

Enfin, on définit une deuxième représentation du Rubik's Cube, qui correspond à la première, mais sous une forme compressée (on expliquera plus tard pourquoi on la nécessite). Pour ce faire, on compresse l'information contenue dans les 4 tableaux dans 4 entiers, grâce à différentes astuces combinatoires.

```c
struct pos_comp {
    uint16_t pc;
    uint16_t oc;
    uint32_t pa;
    uint16_t oa;
};
typedef struct pos_comp pos_comp;
```

Par exemple, le tableau d'orientation des arêtes est un tableau de 0 et de 1, que l'on peut voir comme un nombre en base binaire, et que l'on converti alors en base décimale. Pour les permutations, on utilise un code de Lehmer pour les transformer en des n-uplets de $[\![1,\ 1]\!]\times[\![1,\ 2]\!]\times \ldots \times [\![1,\ n]\!]$, que l'on peut alors voir comme des nombres en base factorielle, eux-aussi convertis en base décimale.

Maintenant que l'on a un Rubik's Cube fonctionnel, on peut passer à l'implémentation pratique des algorithmes. Pour chacun des deux algorithmes, la première étape consiste à générer les heuristiques qui servent à l'exécution d'IDA*, dont les valeurs sont stockées dans des bases de données. J'ai opté pour ces bases de données pour de simple fichiers textes, comportant un numéro par ligne, qui correspond à la valeur de l'heuristique pour une position donnée. Cela nécessite donc de construire des fonctions de hachage parfait pour chacune des bases de données, qui donne l'indice de la ligne à consulter dans le fichier texte pour la position donnée en entrée. Par des méthodes combinatoires complexes, on parvient à construire de telles fonctions de hachage, en les rendant de plus minimales, c'est-à-dire qu'elles n'utilisent pas plus de lignes que nécessaire.

On peut alors générer les valeurs de l'heuristique. J'ai d'abord buté sur ce point, car j'ai dans un premier temps pensé que, puisque les sous-problèmes sont d'une complexité bien moindre, on pourrait les résoudre un à un par force brute : lors de la mise en pratique, j'ai finalement estimé le temps nécessité à quelques millénaires ! Il fallait en fait opter pour une programmation dynamique : on initialise la valeur de l'heuristique au maximum qu'elle peut prendre pour toutes les positions, sauf pour la position résolue pour laquelle on la fixe à 0, puis l'on créé une queue qui ne contient initialement que la position résolue, et qui nous permettra d'actualiser de proche en proche les valeurs de l'heuristique.



Position p_1 Position p_2

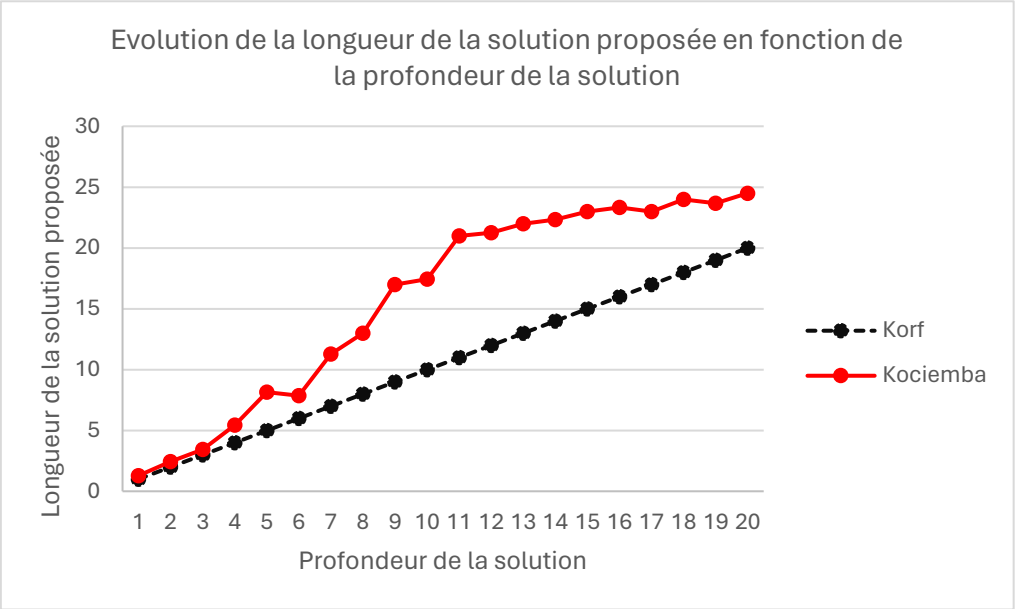
Par exemple, si dans cette situation $heur(p_2) > heur(p_1) + 1$, on peut actualiser la valeur de $heur(p_2)$ à $heur(p_1) + 1$ puis ajouter p_2 à la queue pour voir si cette actualisation modifie la valeur calculée de l'heuristique pour les voisins de p_2.

Une fois les bases de données générées, l'écriture des programmes de Korf et de Kociemba se font sans problème.

IV- Résultats

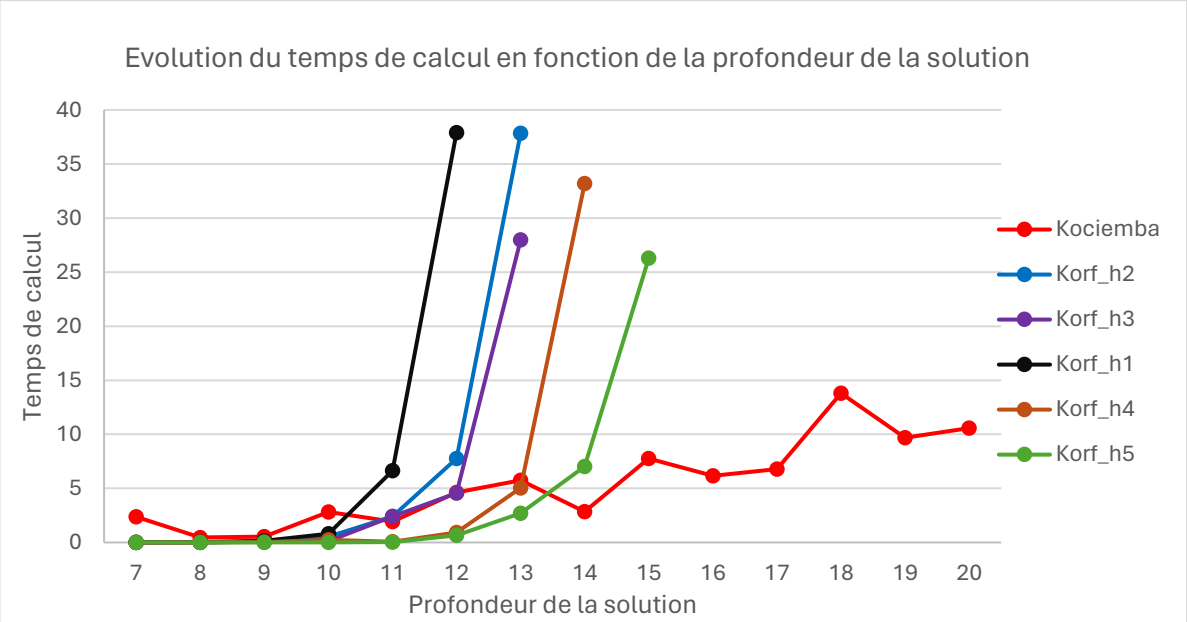
On compare dans cette partie les performances de nos deux implémentations des algorithmes de Korf et de Kociemba.

On peut d'abord comparer ces algorithmes selon la longueur de la solution qu'ils proposent. Les données suivantes ont été générées par des exécutions des algorithmes sur un jeu de données de position à des profondeur différentes .



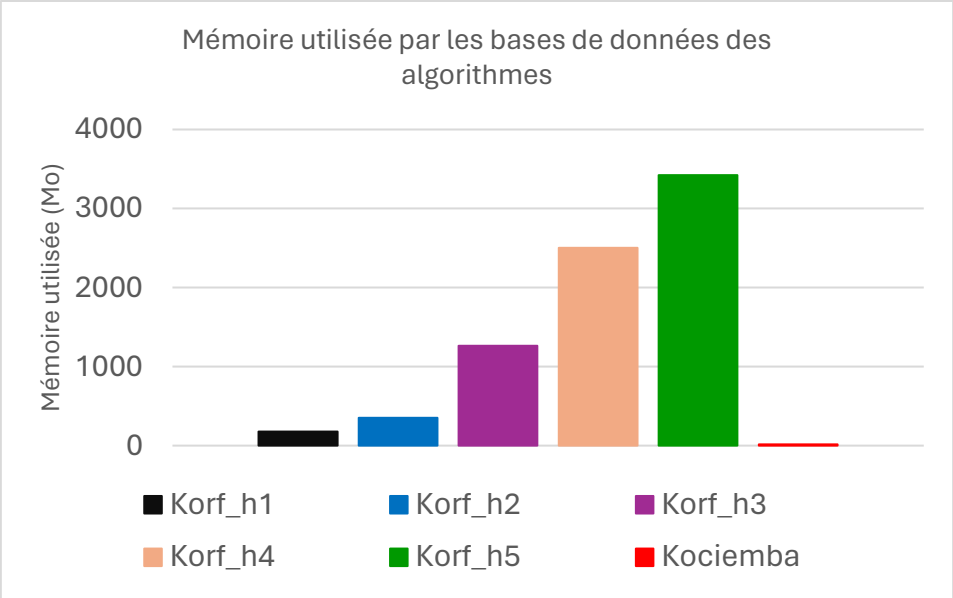
L'algorithme de Korf étant optimal, il renvoie toujours une solution de longueur égale à la profondeur de la position à résoudre. L'algorithme de Kociemba, propose quant à lui une solution presque optimale, en ne dépassant jamais une solution plus longue que 25 mouvements sur notre jeu de données. La différence entre la longueur de la solution proposée par l'algorithme de Kociemba et le minimum requis est particulièrement importante pour des positions d'une profondeur « intermédiaire », à peu près entre 8 et 14, et devient de moins en moins significative pour des positions plus éloignées de la position à résoudre.

A présent, comparons ces algorithmes du point de vue de leur complexité temporelle. Puisqu'ils utilisent des heuristiques compliquées, celle-ci n'est pas calculable dans le cas général, et on analyse ainsi les temps d'exécution en fonction de la profondeur.



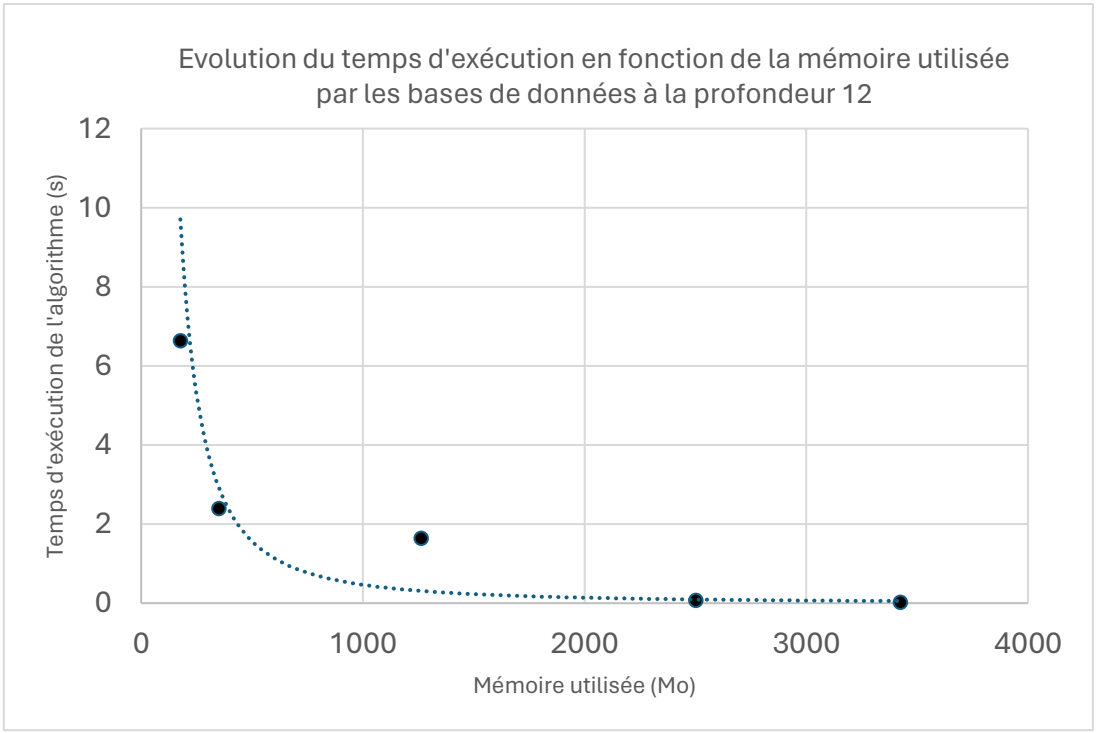
Seul l'algorithme de Kociemba parvient à résoudre n'importe quelle position du Rubik's Cube, et même si son temps d'exécution croit avec la profondeur, celui-ci reste inférieur à 15 secondes même pour les

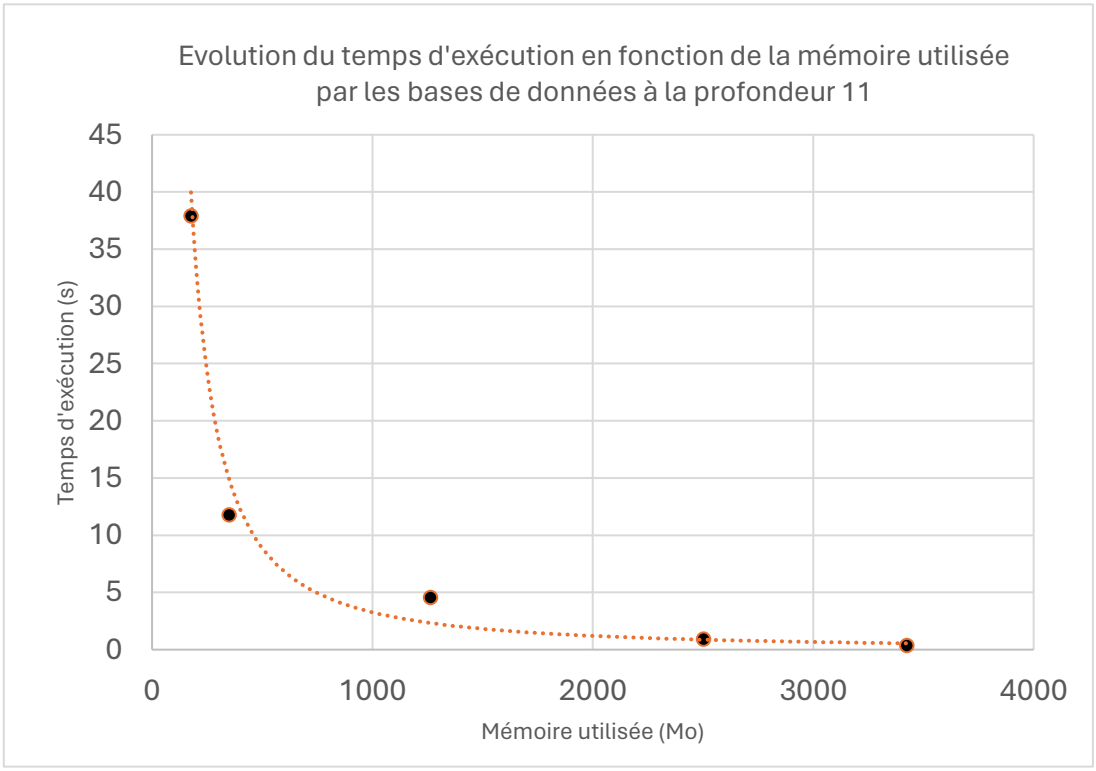
solutions les plus éloignés, ce qui est très raisonnable. On est par contre limité avec l'algorithme de Korf, qui même avec la meilleure heuristique, ne peut garder un temps raisonnable à partir de la profondeur 15 : il fonctionne bien sur les positions peu profondes voire intermédiaires, mais au-delà, son temps d'exécution semble croître de manière exponentielle. On voit nettement l'influence de la quantité de mémoire utilisée par les bases de données sur le temps d'exécution des algorithmes, passant d'une profondeur maximale de résolution de 12 pour l'algorithme utilisant des bases de donnée les plus réduites, à 15 pour celui possédant les bases de données les plus lourdes. Cela nous invite à considérer la complexité spatiale de chacun de ces algorithmes.



Comme voulu, les 5 heuristiques de l'algorithme de Korf correspondent à une mémoire utilisée de plus en plus grande. Ce que l'on remarque surtout, c'est que la quantité de mémoire utilisée par l'algorithme de Kociemba est infime vis-à-vis des algorithmes de Korf, même lorsque l'on considère l'heuristique de Korf la plus simple.

On en vient donc à considérer la conjecture de Korf, qui affirme que le temps d'exécution de l'algorithme de Korf est inversement proportionnel à la mémoire utilisée par les bases de données. J'ai décidé d'essayer de la vérifier expérimentalement à deux profondeurs intermédiaires où tous les algorithmes peuvent résoudre le Rubik's Cube en un temps raisonnable. On obtient bien un profil hyperbolique, mais qui est mieux approché par un n^{α} où α proche de 1,5. Dans tous les cas, l'idée de la conjecture de Korf, qui indique qu'utiliser plus de mémoire permet d'abord de diminuer assez rapidement le temps d'exécution, pour devenir de moins en moins intéressant est vérifiée.





V- Conclusion

Ainsi, les algorithmes de résolution optimale, comme celui de Korf, sont, malgré leur intérêt théorique, limités en pratique car ils ne peuvent s'appliquer qu'à des positions qui ne sont pas trop éloignées de la position résolue. On peut toutefois améliorer leur rapidité en sacrifiant de la mémoire, pour un gain d'abord significatif puis de moins en moins intéressant. L'algorithme de Kociemba, qui relâche la contrainte d'optimalité, fournit quant à lui des solutions dans un temps toujours raisonnable et avec une

consommation de mémoire minime : le fait qu'il soit l'algorithme de résolution informatique du Rubik's Cube le plus répandu ne fait aucune surprise.

Bibliographie

[1] Erik D. Demaine, Sarah Eisenstat, and Mikhail Rudoy. Solving the Rubik's Cube Optimally is NP-complete. In 35th Symposium on Theoretical Aspects of Computer Science (STACS 2018). Leibniz International Proceedings in Informatics (LIPIcs), Volume 96, pp. 24:1-24:13, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2018)

[2] Tomas Rockiki, Herbert Kociemba, Morley Davidson, John Dethridge, The diameter of the Rubik's Cube group is twenty, SIAM J. Discrete Math. 27 (2) (2013) 1082-1105.

[3] Richard E. Korf. Finding optimal solutions to Rubiks cube using pattern databases. In Proc. 19th National Conference on Artificial Intelligence AAAI, 1997, pp. 700-705.

[4] Joseph Culberson, Jonathan Schaeffer, Pattern databases, Comput. Intell. 14 (3) (1998) 318-334.

[5] Herbert Kociemba. The two-phase-algorithm. http://kociemba.org/cube.htm